UNIT – II

Overloading - Inheritance – Files and Stream – Multithreading – Exception Handling

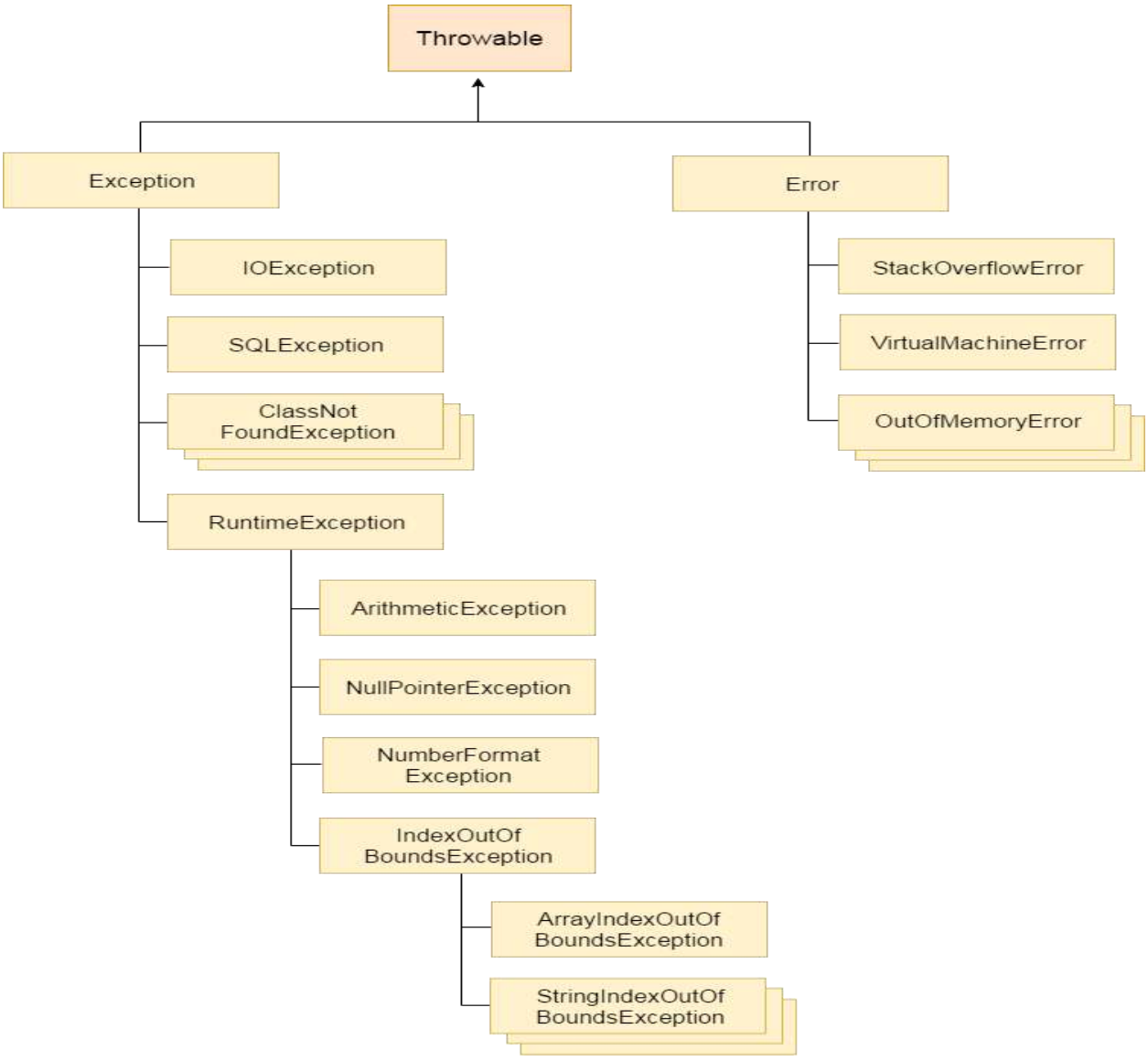
Exception Handling

Exception is an event that **disrupts the normal flow of the program**. It is an object which is thrown at runtime.

Advantage of Exception Handling

The core advantage of exception handling is **to maintain the normal flow of the application**.

Hierarchy of Java Exception classes



Types of Exception

1. Checked Exception
2. Unchecked Exception
3. Error

Difference between checked and unchecked exceptions

1) **Checked Exception**

The classes that extend Throwable class except RuntimeException and Error are known as checked exceptions e.g.IOException, SQLException etc. Checked exceptions are checked at compile-time.

2) **Unchecked Exception**

The classes that extend RuntimeException are known as unchecked exceptions e.g. ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException etc. Unchecked exceptions are not checked at compile-time rather they are checked at runtime.

3) **Error**

Error is irrecoverable e.g. OutOfMemoryError, VirtualMachineError, AssertionError etc.

Java Exception Handling Keywords

1. try
2. catch
3. finally
4. throw
5. throws

<u>try-catch</u>

The try block contains a block of program statements within which an exception might occur. A try block is always followed by a catch block, which handles the exception that occurs in associated try block

Syntax of try catch in java

```
try
{
//statements that may cause an exception
}
```

```
catch (exception(type) e(object))
{
//error handling code
}
```

Finally

A **finally statement** must be associated with a **try statement**. It identifies a block of statements that needs to be executed regardless of whether or not an **exception occurs** within the try block.

Syntax of Finally block

```
try
{
//statements that may cause an exception
}
finally
{
//statements to be executed
}
```

throws

- The **Java throws keyword** is used to declare an exception. It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.

Syntax of java throws

```
return_type method_name() throws exception_class_name{
//method code
}
```

throw

- The Java throw keyword is used to explicitly throw an exception.
- We can throw either checked or uncheked exception in java by throw keyword. The throw keyword is mainly used to throw custom exception. We will see custom exceptions later.

syntax of java throw keyword

throw exception;

Java Custom Exception Or user-defined exception

- If you are creating your own Exception that is known as custom exception or user-defined exception. Java custom exceptions are used to customize the exception according to user need.

Example:

```
class InvalidAgeException extends Exception{
 InvalidAgeException(String s){
  super(s);
 }
}

class TestCustomException1{


    static void validate(int age)throws InvalidAgeException{
            if(age<18)
            throw new InvalidAgeException("not valid");
            else
            System.out.println("welcome to vote");
    }


public static void main(String args[]){
try{
validate(13);
}
catch(Exception m){System.out.println("Exception occured: "+m);}


System.out.println("rest of the code...");
}
}
```

Error Vs Exception

Basis for Comparison	Error	Exception
Basic	An error is caused due to lack of system resources.	An exception is caused because of the code.
Recovery	An error is irrecoverable.	An exception is recoverable.
Keywords	There is no means to handle an error by the program code.	Exceptions are handled using three keywords "try", "catch", and "throw".
Consequences	As the error is detected the program will terminated abnormally.	As an exception is detected, it is thrown and caught by the "throw" and "catch" keywords correspondingly.

throw Vs throws

No.	throw	throws
1)	Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
2)	Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
3)	Throw is followed by an instance.	Throws is followed by class.
4)	Throw is used within the method.	Throws is used with the method signature.
5)	You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException.

Difference between final, finally and finalize

No.	final	finally	finalize
1)	Final is used to apply restrictions on class, method and variable. Final class can't be inherited, final method can't be overridden and final variable value can't be changed.	Finally is used to place important code, it will be executed whether exception is handled or not.	Finalize is used to perform clean up processing just before object is garbage collected.
2)	Final is a keyword.	Finally is a block.	Finalize is a method.

Inheritance

Inheritance in java is a mechanism in which one object acquires all the properties and behaviors of parent object.

Inheritance advantage

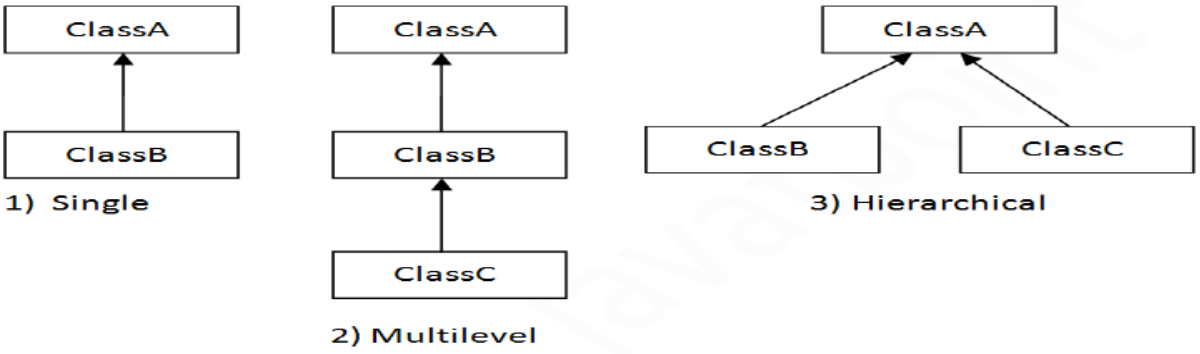
- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

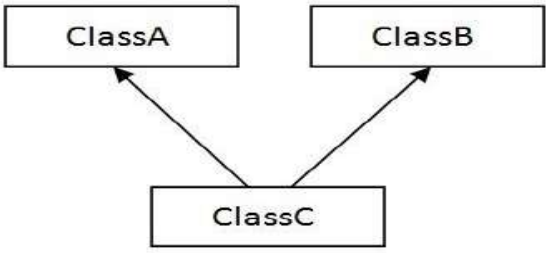
Syntax of Java Inheritance

```
class Subclass-name extends Superclass-name
{
//methods and fields
}
```

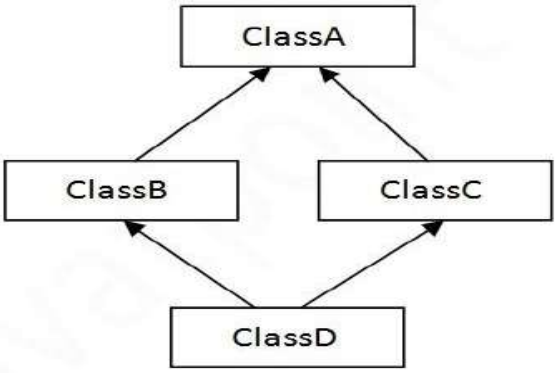
Types of inheritance in java

- **Single Inheritance**-enables a derived class to **inherit** properties and behavior from a **single** parent class
- **Multilevel Inheritance**-If we take the example of above diagram then class C **inherits** class B and class B **inherits** class A which means B is a parent class of C and A is a parent class of B. So in this case class C is implicitly inheriting the properties and method of class A along with B that's what is called **multilevel inheritance**
- **Hierarchical Inheritance -** multiple classes inherits from a single class i.e there is one super class and multiple subclasses.
- **multiple inheritance-inherit** properties of more than one parent class
- **Hybrid inheritance -** a combination of **Single** and **Multiple inheritance**





4) Multiple



5) Hybrid

.

Single Inheritance Example

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class TestInheritance{
public static void main(String args[]){
Dog d=new Dog();
d.bark();
d.eat();
}}
```

Output

```
barking...
eating...
```

Multilevel Inheritance Example

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
public static void main(String args[]){
```

```
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```

Output:

```
weeping...
barking...
eating...
```

Hierarchical Inheritance Example

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
public static void main(String args[]){
Cat c=new Cat();
c.meow();
c.eat();
//c.bark();//C.T.Error
}}
```

output

1. meowing...
2. eating...

Multiple InheritanceExample

```
interface Printable{
void print();
}
interface Showable{
void show();
}
class A7 implements Printable,Showable{
public void print(){System.out.println("Hello");}
public void show(){System.out.println("Welcome");}
public static void main(String args[]){
A7 obj = new A7();
```

```
    obj.print();
    obj.show();
    }
    }
```

Output:

```
Hello
Welcome
```

Hybrid Inheritance Example

```
publicclassClassA
{
publicvoid dispA()
{
System.out.println("disp() method of ClassA");
}
}
publicinterfaceInterfaceB
{
publicvoid show();
}
publicinterfaceInterfaceC
{
publicvoid show();
}
publicclassClassDimplementsInterfaceB,InterfaceC
{
publicvoid show()
{
System.out.println("show() method implementation");
}
publicvoid dispD()
{
System.out.println("disp() method of ClassD");
}
publicstaticvoid main(String args[])
{
      ClassD d =newClassD();
      d.dispD();
      d.show();
}
}
```

Output :

```
disp() method of ClassD
show() method implementation
```

Method Overloading (or) Compile time Polymorphism

If a class has multiple methods having same name but different in parameters, it is known as **Method Overloading**.

Advantage of method overloading-*increases the readability of the program*.

Different ways to overload the method

1. By changing number of arguments
2. By changing the data type

Method Overloading: changing no. of arguments

In this example, we have created two methods, first add() method performs addition of two numbers and second add method performs addition of three numbers.

In this example, we are creating static methods so that we don't need to create instance for calling methods.

```
class Adder{
static int add(int a,int b){return a+b;}
static int add(int a,int b,int c){return a+b+c;}
}
class TestOverloading1{
public static void main(String[] args){
System.out.println(Adder.add(11,11));
System.out.println(Adder.add(11,11,11));
}}
```

Output:

```
22
33
```

Method Overloading: changing data type of arguments

In this example, we have created two methods that differs in data type. The first add method receives two integer arguments and second add method receives two double arguments.

```
class Adder{
static int add(int a, int b){return a+b;}
static double add(double a, double b){return a+b;}
}
class TestOverloading2{
public static void main(String[] args){
System.out.println(Adder.add(11,11));
System.out.println(Adder.add(12.3,12.6));
}}
```

Output:

```
22
24.9
```

Constructor Overloading in Java

Constructor overloading is a technique in Java in which a class can have any number of constructors that differ in parameter lists.The compiler differentiates these constructors by taking into account the number of parameters in the list and their type.

Example of Constructor Overloading

```
class Student5{
   int id;
   String name;
   int age;
   Student5(int i,String n){
   id = i;
   name = n;
   }
   Student5(int i,String n,int a){
   id = i;
   name = n;
   age=a;
   }
   void display(){System.out.println(id+" "+name+" "+age);}

public static void main(String args[]){
Student5 s1 = new Student5(111,"Karan");
Student5 s2 = new Student5(222,"Aryan",25);
s1.display();
s2.display();
}
}
```

Output:

```
111 Karan 0
222 Aryan 25
```

MULTITHREADING

- Java provides built-in support for **multithreaded programming.**
- Multithreaded programs extend the idea of multitasking by taking it one level lower: individual programs will appear to do multiple tasks at the same time.
- A multithreaded program contains two or more parts that can runconcurrently.
- Each part of such a program is called a **thread**, and each thread defines a separate path of execution.
- Programs that can run more than one thread at once are said to be multithreaded.
- Thus, multithreading is a specialized form of multitasking.

PROGRAM

```
import java.util.*;
import java.io.*;
import java.lang.*;
class A extends Thread
{
	public void run()
	{
		System.out.println("Start A");
		for(int i=1;i<=5;i++)
		 System.out.println("Thread A: "+i);
		System.out.println("Exit A");
	}
}

class B extends Thread
{
	public void run()
	{
		System.out.println("Start B");
		for(int i=1;i<=5;i++)
		 System.out.println("Thread B: "+i);
		System.out.println("Exit B");
	}
}

class ThreadTest
{
	public static void main(String args[])
	{
		A a=new A();
		Thread t1=new Thread(a);
		Thread t2=new Thread(new B());
```

```
                t1.start();
                t2.start();
        }
}
```

OUTPUT

```
Start A
Start B
Thread B: 1
Thread B: 2
Thread B: 3
Thread B: 4
Thread B: 5
Exit B
Thread A: 1
Thread A: 2
Thread A: 3
Thread A: 4
Thread A: 5
Exit A
```

PROGRAM

```
import java.util.*;
import java.io.*;
import java.lang.*;
class A extends Thread
{
        int n=10;
        String str="";
        A(String s)
        {
                str=s;
        }
        public void run()
        {
                for(int i=1;i<=n;i++)
                {
                 try
                 {
                        System.out.println(str+i);
                        Thread.sleep(20);
                 }
                 catch(Exception e){}
```

```
                }
        }
}

class Threading
{
        public static void main(String args[])
        {
                A a=new A("Issuing ticket");
                A b=new A("Showing seat");
                Thread t1=new Thread(a,"Person A");
                Thread t2=new Thread(b,"Person B");
                t1.start();
                t2.start();
        }
}
```

OUTPUT

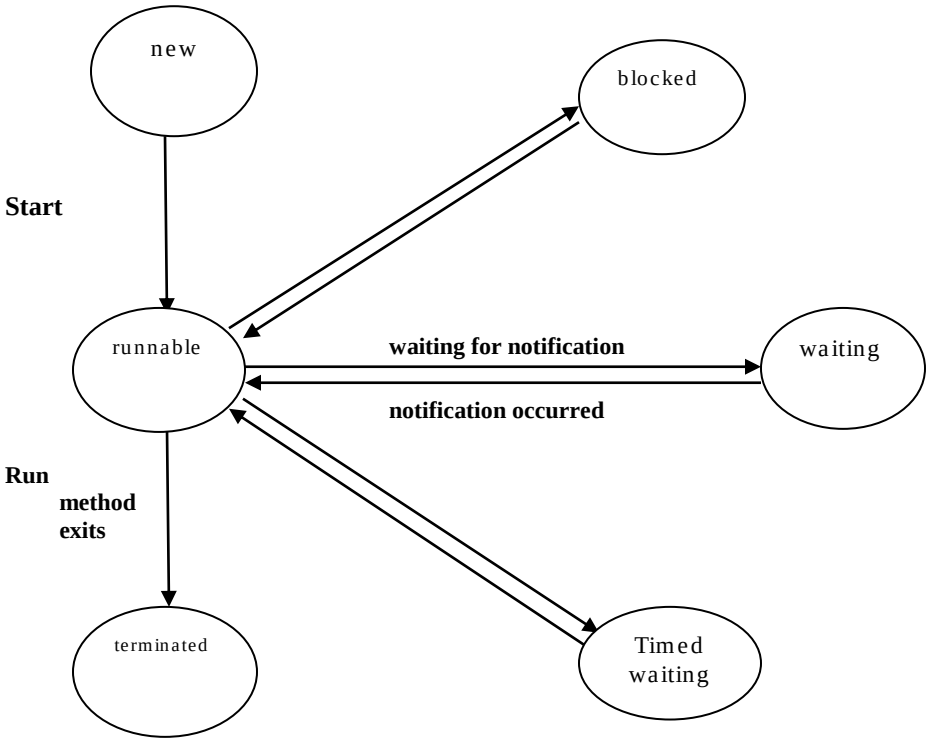
```
Issuing ticket1
Showing seat1
Showing seat2
Issuing ticket2
Issuing ticket3
Showing seat3
Issuing ticket4
Showing seat4
Issuing ticket5
Showing seat5
Showing seat6
Issuing ticket6
Showing seat7
Issuing ticket7
Issuing ticket8
Showing seat8
Issuing ticket9
Showing seat9
Issuing ticket10
Showing seat10
```

THREADS CLASS METHODS

start()	**key** to start a method and moves a thread to runnable state.
run()	**heart** of any thread created in java and moves a thread to runnable state.
stop()	**stops** a thread from running state and moves a thread to **dead** state.
sleep()	**suspends** the thread to specified no of milliseconds.
suspend()	suspends the execution of a thread until the **resume()** is called.
wait()	suspends the execution of a thread until the **notify()** is called.
getName()	obtain the **name** of a Thread.
getPriority()	obtain the **priority** of a Thread.

THREAD STATES

- Threads can be in one of six states:
    1. **New**
    2. **Runnable**
    3. **Blocked**
    4. **Waiting**
    5. **Timed waiting**
    6. **Terminated**



NEW THREADS

- When you create a thread with the new operator—for example, new Thread(r)—the thread is not yet running

RUNNABLE THREADS

- Once you invoke the start method, the thread is in the **runnable**state.
- A runnable thread may or may not actually be running

BLOCKED AND WAITING THREADS

- When a thread is blocked or waiting, it is temporarily inactive.
- It doesn't execute any code and it consumes minimal resources

TERMINATED THREADS

- A thread is terminated for one of two reasons:
  - It dies a natural death because the run method exits normally.
  - It dies abruptly because an uncaught exception terminates the run method.

void join()	waits for the specified thread to terminate.
void join(long millis)	waits for the specified thread to die or for the specified number of milliseconds to pass.
Thread.StategetState() 5.0	gets the state of this thread; one of NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, or TERMINATED.
void stop()	stops the thread. This method is deprecated.
void suspend()	suspends this thread's execution. This method is deprecated.
void resume()	resumes this thread. This method is only valid after suspend() has been invoked.

THREAD SYNCHRONIZATION

- When two or more threads need access to a shared resource, they need some way to ensure that the resource will be used by only one thread at a time.
- The process by which this is achieved is called **synchronization.**
- While a thread is inside a synchronized method, all other threadsthat try to call it (or any other synchronized method) on the same instance have to wait.

- **RACE CONDITION**

- Depending on the order in which the data were accessed, corrupted objects can result. Such a situation is often called a race condition

- **LOCK OBJECTS**
  - The Java language provides a synchronized keyword for this purpose, introduced the ReentrantLock class.
  - The synchronized keyword automatically provides a lock as well as an associated "condition," which makes it powerful and convenient for most cases that require explicit locking.

- **READ/WRITE LOCKS**
  - The java.util.concurrent.locks package defines two lock classes, the ReentrantLock and the ReentrantReadWriteLock class, useful when there are many threads that read from a data structure and fewer threads that modify it.
  - In that situation, it makes sense to allow shared access for the readers.

PROGRAM

```
import java.util.*;
import java.io.*;
import java.lang.*;

class Reservation extends Thread
{
        int available=1;
        int wanted;
        Reservation(int w){wanted=w;}
        public void run()
        {
            synchronized(this)
           {
                System.out.println("\nAvailable tickets: "+available);
                String s = Thread.currentThread().getName();
                if(wanted<=available)
                {

                        System.out.println(s+" reserved "+wanted+" ticket");
                        try
                        {
                                available=available-wanted;
                                Thread.sleep(20);
                        }
                        catch(Exception e){}
                 }
```

```
                else
                        System.out.println("Sorry!!No tickets for " +s);
            }
        }
}

class ThreadTesting
{
        public static void main(String args[])
        {
                Reservation a=new Reservation(1);
                Thread t1=new Thread(a,"Person A");
                Thread t2=new Thread(a,"Person B");
                System.out.println("\tReservation");
                t1.start();
                t2.start();
        }
}
```

OUTPUT

```
    Reservation

Available tickets: 1
Person A reserved 1 ticket

Available tickets: 0
Sorry!!No tickets for Person B
```

Files and Stream

- A stream is a **sequence of data**.In Java a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.
- In java, 3 streams are created for us automatically
    1) **System.out:** standard output stream
    2) **System.in:** standard input stream
    3) **System.err:** standard error stream

Types of Streams

The java.io package contains a large number of stream classes that provide capabilities for processing all types of data. These classes may be categorized into two groups based on the data type on which they operate.

- **Byte stream classes**
- **Character stream classes**

Character Stream

In Java, characters are stored using Unicode conventions. Character stream automatically allows us to read/write data character by character. For example FileReader and FileWriter are character streams used to read from source and write to destination.

Byte Stream

Byte streams process data byte by byte (8 bits). For example FileInputStream is used to read from source and FileOutputStream to write to the destination.

File-Reading:

- **Commands for opening the file for reading:**

  FileReader fr = new *FileReader*("input.txt");
  BufferedReader br = new *BufferedReader*(fr);
- **Command for reading one line from the opened file:**

  String str = br.*readLine*();
- **Command for closing the opened file:**

  br.*close*();

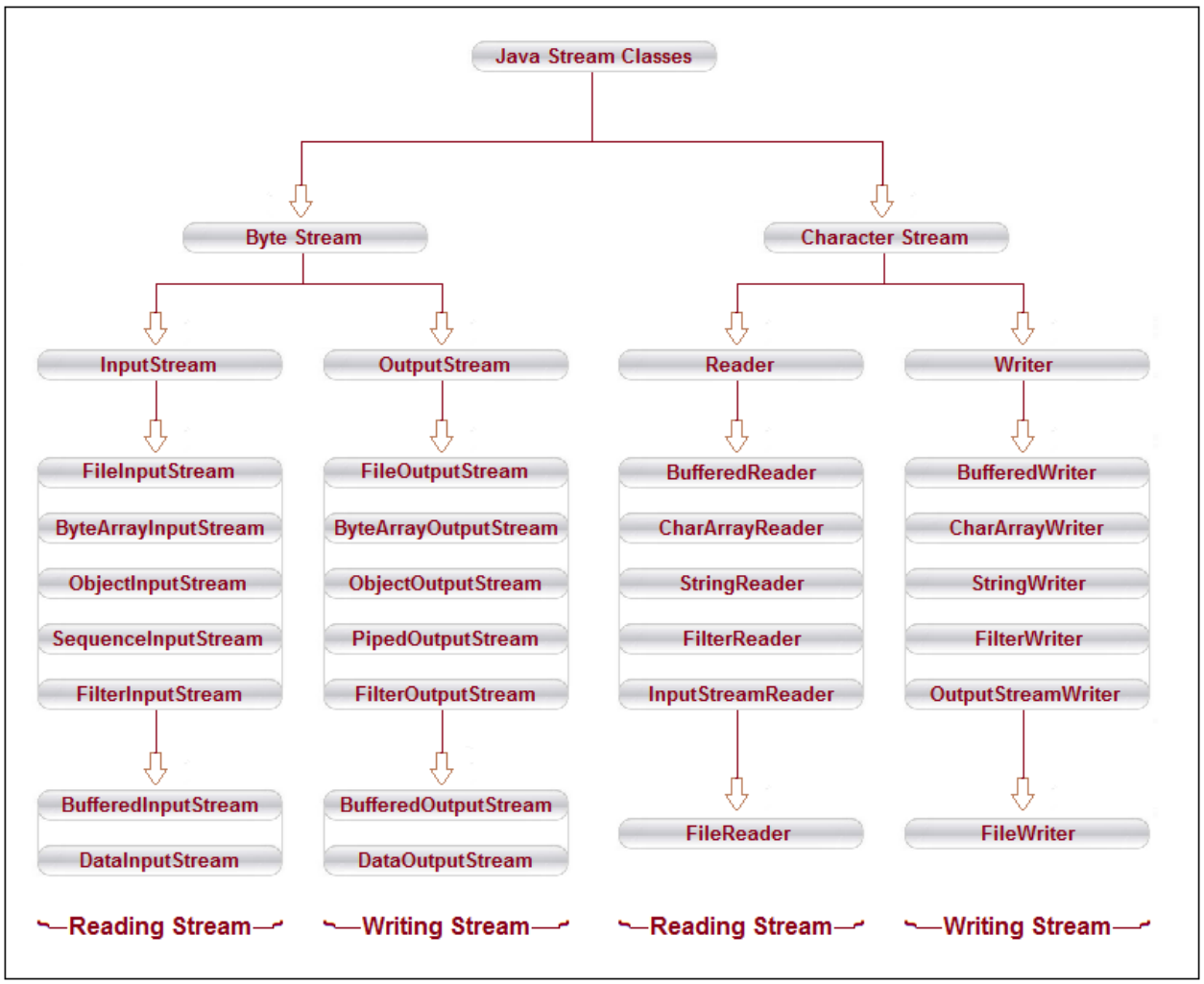
File-Writing:

- **Commands for opening the file for writing:**

  FileWriter fw = new *FileWriter*("output.txt");
  BufferedWriter bw = new *BufferedWriter*(fr);
- **Command for writing a String to the opened file:**

  bw.*write*(str); // str is a String
- **Command for closing the opened file:**

  bw.*close*();



Java FileInputStream example 1: read single character

```
import java.io.FileInputStream;
public class DataStreamExample {
        public static void main(String args[]){
                try{
                FileInputStream fin=new FileInputStream("D:\\testout.txt");
                int i=fin.read();
                System.out.print((char)i);

                fin.close();
                }catch(Exception e){System.out.println(e);}
        }
        }
```

Java FileOutputStream example 2: write string

```
import java.io.FileOutputStream;
public class FileOutputStreamExample {
   public static void main(String args[]){
        try{
          FileOutputStream fout=new FileOutputStream("D:\\testout.txt");
          String s="Welcome to javaTpoint.";
          byte b[]=s.getBytes();//converting string into byte array
          fout.write(b);
          fout.close();
          System.out.println("success...");
         }catch(Exception e){System.out.println(e);}
    }
}
```

Output:

```
Success...
```

testout.txt

```
Welcome to javaTpoint.
```